GymLandT

Gym & Fitness Membership Management System

Project Documentation

Field	Detail
Project Title	GymLandT — Gym & Fitness Membership Management System
Course Name	Advanced JavaScript Backend Frameworks (Node.js & Express JS)
Semester	5th Semester
Department	ADSE
Section / Batch	5 BTCS AIML A
Institution	Christ University
Submission	September 09, 2026

Team Details

S.No	Student Name	Roll Number	Department
1	Harshdeep Sharma	2460370	ADSE
2	Laya Shaju	2460397	ADSE
3	Abhishan Francis	2462835	ADSE

GitHub Repository

Repository Link: https://github.com/Harshxox/L_AND_T_PROJECT_CIA-3_PROJECT_DEVELOPMENT

The repository is public and contains the complete GymLandT source code, documentation, and the consolidated Postman collection.

Table of Contents

1. Introduction

1.1 Overview

The Gym & Fitness Membership Management System (GymLandT) is a full-stack web application that digitizes the day-to-day operations of a gym or fitness branch — replacing manual, register-based tracking of memberships, class schedules, bookings, and attendance with a role-based online platform.

1.2 Problem Statement

Traditional gyms often rely on paper registers or basic spreadsheets to track memberships, class attendance, and trainer schedules. This leads to:

- No automated tracking of membership expiry, leading to lapsed members being allowed to continue using facilities.
- No structured way to manage class capacity, resulting in overbooking or manual turn-aways. ● No systematic attendance history for members or performance data for admins.
- No self-service access for members to view their own membership, bookings, or attendance.

1.3 Objectives

- Provide members a self-service portal to register, purchase/renew memberships, browse and book classes, and track their own attendance and workout plans.
- Provide trainers a portal to manage their assigned classes, record attendance, and maintain workout/diet notes for their members.
- Provide branch admins full administrative control over plans, trainers, classes, and branch-wide reporting.
- Enforce real business rules programmatically (membership expiry, class capacity, waitlist promotion) rather than relying on manual staff judgment.
- Demonstrate a complete, working system — not a mockup — with persistent data, authentication, and rolebased authorization enforced at the API level.

1.4 Scope

The system covers registration and authentication, membership plan management, membership purchase/renewal/expiry, trainer management, class scheduling, class booking with waitlisting, attendance check-in, workout/diet plan notes, expiry notifications, a self-service member dashboard, and branch-admin reporting (attendance trends, plan popularity, renewal rate).

2. System Requirements

2.1 Functional Requirements

#	Requirement
1	Members can register, authenticate, and manage their own profile
2	Members can purchase and renew membership plans
3	System tracks membership status (ACTIVE / EXPIRED / CANCELLED / PENDING) automatically

4	Admins can manage membership plans, trainers, and classes

#	Requirement
5	Members can book classes; system enforces capacity
6	Members are waitlisted when a class is full, with automatic promotion on cancellation
7	Trainers can record attendance for their assigned classes
8	Trainers can maintain workout/diet plan notes per member
9	System generates membership-expiry reminder notifications
10	Admins can view attendance, membership popularity, and renewal-rate reports
11	Role-based access control restricts each endpoint to authorized roles

2.2 Non-Functional Requirements

- Security — passwords hashed with bcrypt, JWT-based stateless authentication, no secrets hard-coded, server-side validation on every input.
- Reliability — centralized error handling; no unhandled promise rejection may crash the server; invalid input returns clean JSON errors, not stack traces or HTML.
- Usability — responsive frontend, clear loading/error states, role-appropriate dashboards.
- Maintainability — layered MVC/service architecture keeps business logic out of route handlers.

2.3 Hardware / Software Requirements

- Backend: Node.js, Express.js
- Database: MongoDB (local or MongoDB Atlas)
- ODM: Mongoose
- Authentication: JSON Web Tokens (JWT), bcrypt/bcryptjs
- Validation: Joi / express-validator
- Frontend: React (Vite + Tailwind CSS)
- API Testing: Postman
- Version Control: Git / GitHub

3. System Architecture

3.1 Architectural Style

The system follows a layered client-server architecture with clear separation of concerns:

```
Frontend (React)
    |  HTTPS / REST
Express Routes
    |
Middleware (Auth -> Role -> Validation)
    |
```

```
Controllers
    |
Services (business logic)
    |
Mongoose Models
    |
MongoDB
```

Business logic is deliberately kept out of route files and controllers where possible, and placed in a dedicated services layer (membershipService, bookingService, waitlistService, notificationService, reportService). This keeps controllers thin — parse request, call service, format response — and makes business rules independently testable.

3.2 Authentication Flow

```
Client submits credentials
    |
authController validates & compares bcrypt hash
    |
JWT issued (signed with JWT_SECRET, expires per JWT_EXPIRES_IN)
    |
Client stores token, attaches Authorization: Bearer <token>
    |
auth middleware verifies token (401 if missing/invalid/expired)
    |
role middleware checks role against allowed roles (403 if disallowed)
    |
Controller executes
```

3.3 Project Folder Structure

```
project-root/
|-- config/            DB connection config
|-- controllers/       Request handlers per module
|-- routes/            Express route definitions
|-- middleware/        auth.js, role.js, validate.js, errorHandler.js
|-- services/          Business logic layer
|-- validators/        Request validation schemas
|-- utils/             Shared helpers (jwt.js, seeder.js)
|-- models/            Mongoose schemas
|-- tests/             Automated test suite
|-- postman/           Exported Postman collection
|-- frontend-react/    React (Vite + Tailwind) client
|-- .env / .env.example
|-- package.json
|-- server.js
```

4. User Roles & Permissions

The system implements exactly three roles, enforced server-side via role middleware (requireRole), so that access restrictions cannot be bypassed by manipulating the frontend.

Role	Key Permissions
MEMBER	Register/login, view profile, purchase/renew membership, browse and book classes, join waitlists, cancel own bookings, view own attendance history, view own workout/diet notes, view notifications
TRAINER	Login, view/update own profile, view assigned classes, record attendance for assigned classes, create/update workout plans for assigned members, view relevant member info
Role	**Key Permissions**
BRANCH ADMIN	Login, manage membership plans, manage trainers, manage classes, view all bookings, manage/inspect waitlists, view all reports, manage branch-level data

A member cannot access admin or trainer-only endpoints even by calling the API directly — every protected route validates the authenticated user's role, not just hides a button in the UI.

5. Functional Modules

Module 1 — Registration & Authentication

Public registration defaults new accounts to the MEMBER role. Passwords are hashed with bcrypt before storage; plaintext passwords are never persisted or returned in any API response. Login validates credentials and issues a signed JWT. Protected routes reject missing, invalid, or expired tokens with 401.

Module 2 — Membership Plan Management

Admins create and manage plans (e.g. Basic, Standard, Premium) with durationMonths, price, and status. Business rules: duration must be positive, price cannot be negative, and inactive plans cannot be newly purchased.

Module 3 — Membership Purchase & Expiry Tracking

On purchase, the system validates the plan is active, sets a start date, and calculates the expiry date from durationMonths. Membership status (ACTIVE / EXPIRED / CANCELLED / PENDING) is maintained automatically — an expired membership cannot remain marked ACTIVE, and expired members are blocked from member-only actions elsewhere in the system. Renewal correctly extends the membership dates and increments a renewal counter.

Module 4 — Trainer Profile Management

Admins create, view, update, and activate/deactivate trainer profiles, each linked to a user account with the TRAINER role.

Module 5 — Class Schedule Management

Classes are defined with a trainer, date/time window, category, and capacity. Validation ensures capacity is positive, end time is after start time, and the assigned trainer exists and is active. bookedCount is kept consistent with actual confirmed bookings rather than tracked independently.

Module 6 — Class Booking Engine

The booking workflow runs a strict sequence: authenticate, verify active membership, verify class exists/isn't cancelled/hasn't started, verify the member isn't already booked, check capacity, create the booking, increment bookedCount. Duplicate active bookings for the same class by the same member are prevented.

Module 7 — Attendance Check-In

Records gym visits and class attendance. A member must have an active membership to check in. Trainers may only record attendance for classes they are assigned to; duplicate check-ins for the same class/date are prevented.

Module 8 — Waitlist for Full Classes

When a class is at capacity, a booking attempt places the member on a waitlist (WAITING / PROMOTED / CANCELLED) instead of failing outright, with position tracked and duplicate entries prevented. When a confirmed booking is cancelled, the system automatically promotes the earliest waiting member with a still-valid active membership into a confirmed booking, increments bookedCount, and updates the waitlist entry's status — implemented as real, testable business logic rather than a placeholder feature.

Module 9 — Diet / Workout Plan Notes

Trainers create and update workout/diet notes for their assigned members. Members can view but not edit their own plans; a trainer cannot modify notes for members not assigned to them.

Module 10 — Renewal & Expiry Notifications

An internal service scans memberships and generates reminder notifications at 7, 3, and 1 day(s) before expiry, with duplicate-prevention logic so the same milestone isn't notified twice. External SMS/email delivery is out of scope — the notification record itself is persisted and surfaced in-app.

Module 11 — Member Self-Service Dashboard

Displays the member's current plan, status, expiry date, and days remaining; upcoming and available classes; attendance summary and history; current workout plans; and unread notifications — all sourced from live API data, not hard-coded.

Module 12 — Branch Admin Reports

Three MongoDB-aggregation-driven reports: attendance trends (total, by day, by class), membership plan popularity (purchases per plan, most popular plan), and renewal rate (renewed memberships / memberships eligible for renewal).

Module 13 — Role-Based Access Control

A reusable requireRole middleware supports single or multiple allowed roles per route, applied consistently across all controllers so authorization is enforced independently of whatever the frontend does or doesn't show.

6. Database Design

6.1 Collections

Collection	Purpose
users	Account credentials and role
membershipPlans	Plan definitions (name, duration, price, status)
memberships	A member's purchased plan instance, with dates and status
trainers	Trainer profile data, linked to a user

classes	Scheduled class sessions
bookings	A member's booking of a class
attendance	Check-in records
waitlists	Waitlist entries for full classes
workoutPlans	Trainer-authored notes per member
notifications	Expiry reminders and other alerts

6.2 Key Relationships (ObjectId references)

- memberships.memberId -> users, memberships.planId -> membershipPlans ● trainers.userId -> users

- classes.trainerId -> trainers
- bookings.classId -> classes, bookings.memberId -> users
- attendance.memberId -> users, attendance.classId -> classes (where applicable)
- waitlists.classId -> classes, waitlists.memberId -> users
- workoutPlans.memberId -> users, workoutPlans.trainerId -> trainers
- notifications.userId -> users, notifications.relatedMembershipId -> memberships

References are used rather than embedding, since these entities are independently queried and shared across modules (e.g. a class is referenced by both bookings and attendance).

6.3 Indexes

Collection	Index	Purpose
users	email (unique)	Prevent duplicate accounts
memberships	memberId, status, endDate	Fast lookup of a member's memberships and expiry scans
classes	trainerId, date	Fast schedule lookups
bookings	classId, memberId (+compound)	Fast booking lookups, duplicate-booking prevention
attendance	memberId, date	Attendance history queries
waitlists	classId, memberId	Waitlist ordering and duplicate prevention

7. API Design

7.1 Response Format

Success response:

```
{ "success": true, "message": "Operation successful", "data": {} }
```

Error response:

```
{ "success": false, "message": "Meaningful error message", "errorCode": "VALIDATION_ERROR" }
```

7.2 HTTP Status Code Convention

Code	Meaning
200	Successful read/update
201	Successful creation
400	Validation failure

401	Authentication failure
403	Authorization failure
404	Resource not found

Code	Meaning
409	Business-rule conflict
500	Unexpected server error

7.3 Endpoint Summary

Module	Endpoints
Auth	POST /api/auth/register, POST /api/auth/login, GET /api/auth/me
Plans	GET/POST /api/plans, GET/PUT/DELETE /api/plans/:id
Memberships	POST /api/memberships, GET /api/memberships/my, GET /api/memberships/:id, POST /api/memberships/:id/renew
Trainers	GET/POST /api/trainers, GET/PUT/DELETE /api/trainers/:id
Classes	GET/POST /api/classes, GET/PUT/DELETE /api/classes/:id
Bookings	POST /api/classes/:id/book, GET /api/bookings/my, DELETE /api/bookings/:id
Attendance	POST /api/attendance/checkin, GET /api/attendance/my, GET /api/attendance/member/:memberId, GET /api/attendance/class/:classId
Waitlist	POST /api/classes/:id/waitlist, GET /api/classes/:id/waitlist, DELETE /api/waitlist/:id
Workout Plans	POST /api/workout-plans, GET /api/workout-plans/my, GET /api/workout-plans/member/:memberId, PUT /api/workout-plans/:id
Notifications	GET /api/notifications, PATCH /api/notifications/:id/read
Reports	GET /api/admin/reports/attendance, GET /api/admin/reports/membership-plans, GET /api/admin/reports/renewals

A full request/response reference with sample payloads is available in the exported Postman collection at postman/Gym-Management-API.postman_collection.json.

8. Business Rules Implemented

1. Duplicate email registration is rejected.
2. Incorrect password is rejected without revealing whether the email exists.

3. Missing/invalid/expired JWT returns 401 on protected routes.
4. Members cannot access admin-only endpoints; trainers cannot access admin-only endpoints — enforced server-side.
5. Inactive membership plans cannot be purchased.
6. Expired memberships block member-only actions (booking, check-in).
7. Duplicate class bookings by the same member are rejected.
8. A full class routes new booking attempts to the waitlist instead of allowing overbooking.
9. Cancelling a confirmed booking automatically promotes the earliest eligible waitlisted member.
10. Cancelled classes cannot be booked or waitlisted.
11. Duplicate attendance check-ins for the same class/date are prevented.
12. Duplicate waitlist entries for the same member/class are prevented.
13. Renewal correctly extends membership dates rather than creating an overlapping duplicate membership.
14. Expiry notifications are not duplicated across repeated job runs.
15. Trainers cannot modify workout plans for members not assigned to them.
16. Invalid MongoDB ObjectIds are handled gracefully (400/404) rather than crashing the server.

9. Security Implementation

- Password security: bcrypt/bcryptjs hashing; plaintext passwords are never stored or logged.
- Authentication: stateless JWT, signed with a secret loaded from environment variables, with a configurable expiry.
- Authorization: role-based middleware applied per route, independent of any frontend UI restriction.
- Input validation: every incoming request is validated server-side (Joi/express-validator) — required fields, formats, numeric ranges, dates, ObjectIds — since frontend validation alone is not trusted.
- Environment configuration: MongoDB URI, JWT secret, and other configuration are loaded from .env (excluded from version control) rather than hard-coded; .env.example documents required variables without exposing real values.
- Error handling: centralized error middleware prevents stack traces or internal details from reaching API clients, converts Mongoose validation errors into clean responses, and maps duplicate-key errors to 409.
- CORS: configured to restrict which origins may call the API.

10. Testing

10.1 Approach

Automated backend tests cover authentication, authorization, membership lifecycle, class booking, waitlist promotion, attendance, and reporting endpoints. The test suite is implemented with Jest and Supertest.

10.2 Representative Test Cases

Area	Test Case	Expected Result

Auth	Register with existing email	409/400 rejection
Auth	Login with wrong password	401
Authorization	Member calls admin-only route	403
Membership	Purchase inactive plan	409/400 rejection
Membership	Renew existing membership	Dates extended correctly
Classes	Create class with invalid capacity	400 rejection
Bookings	Book already-booked class twice	409 rejection
Bookings	Book a full class directly	Rejected / routed to waitlist
Area	**Test Case**	**Expected Result**
Waitlist	Cancel confirmed booking on full class	Next waitlisted member autopromoted
Attendance	Duplicate check-in same class/date	Rejected
Reports	Attendance report after seeding new data	Reflects updated live data, not cached

10.3 Manual API Testing

The Postman collection (postman/Gym-Management-API.postman_collection.json) covers happy-path requests, validation failures, authentication/authorization failures, not-found cases, and business-rule conflicts across all modules, using collection variables ({{baseUrl}}, {{token}}) with the JWT auto-captured from the login request's test script.

11. Frontend

The client (React + Vite + Tailwind CSS) provides role-specific interfaces:

- Login / Register
- Member Dashboard — membership status, upcoming classes, attendance summary, workout plans, notifications
- Trainer Dashboard — assigned classes, attendance management, workout plan authoring
- Admin Dashboard — summary cards (total members, active memberships, expiring memberships, trainers, today's classes/attendance), plan/trainer/class management, reports
- Classes, My Bookings, Attendance, Workout Plans, Notifications — member-facing feature pages
- Trainer Management, Admin Reports — admin-facing management pages

The JWT is attached to every protected API request; expired/invalid tokens redirect to login. UI controls are hidden per role for usability, but authorization is always re-checked server-side — the frontend restriction is a convenience, not the actual security boundary.

Screenshots of each major screen are included in Section 15 — Screenshots of Final Output.

12. Deployment / How to Run

Backend

```
git clone <repo-
url> cd project-
root npm install
cp .env.example .env   # fill in MONGODB_URI, JWT_SECRET, JWT_EXPIRES_IN, PORT
npm run seed           # populate demo data
npm start              # or: node server.js
```

Frontend (React)

```
cd frontend-react
npm install
npm run dev            # served at http://localhost:5173 by default
```

Seed data includes one admin (admin@gym.com), at least two trainers, at least five members, three membership plans (Basic/Standard/Premium), five or more classes across different trainers/times/capacities, and a mix of active/expired memberships, bookings, attendance history, workout plans, and notifications — enough to demonstrate the full workflow immediately after seeding.

13. Demonstration Workflow

The system supports a complete end-to-end walkthrough: admin creates a plan, trainer, and class; member registers, logs in, and purchases a membership; member browses and books a class; trainer records attendance; member views their attendance history; trainer adds workout plan notes; member views them; the class is filled to capacity and a new member is waitlisted; a confirmed booking is cancelled and the waitlisted member is automatically promoted; an expiry notification is generated for a soon-to-expire membership; admin reviews attendance trends, plan popularity, and renewal-rate reports.

14. Sample Code

The following excerpts from the working codebase illustrate the Mongoose data model and the role-protected admin API described above.

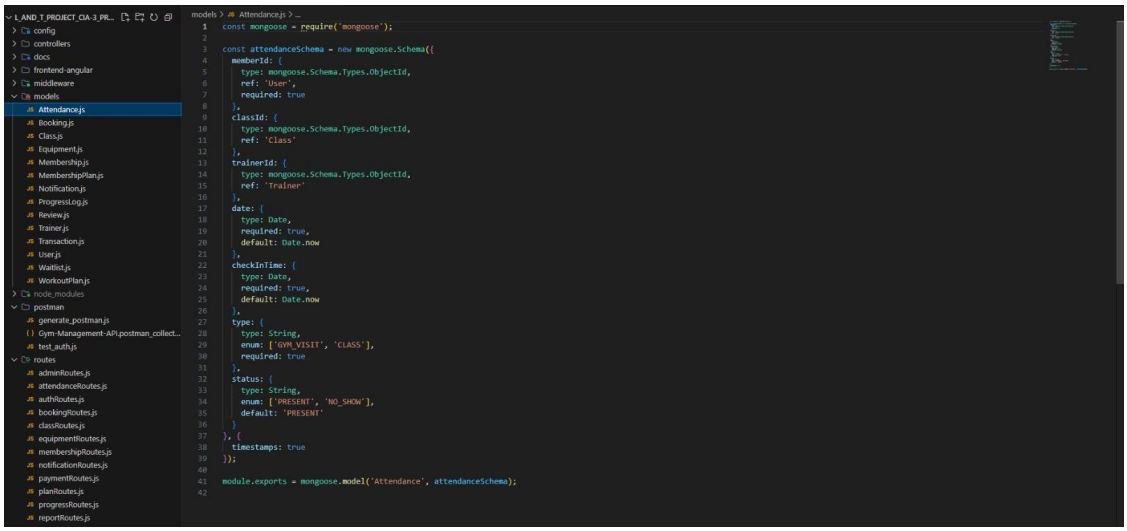

Fig. 14.1 – models/Attendance.js — attendance schema (member/class/trainer references, visit type, present/no-show status)

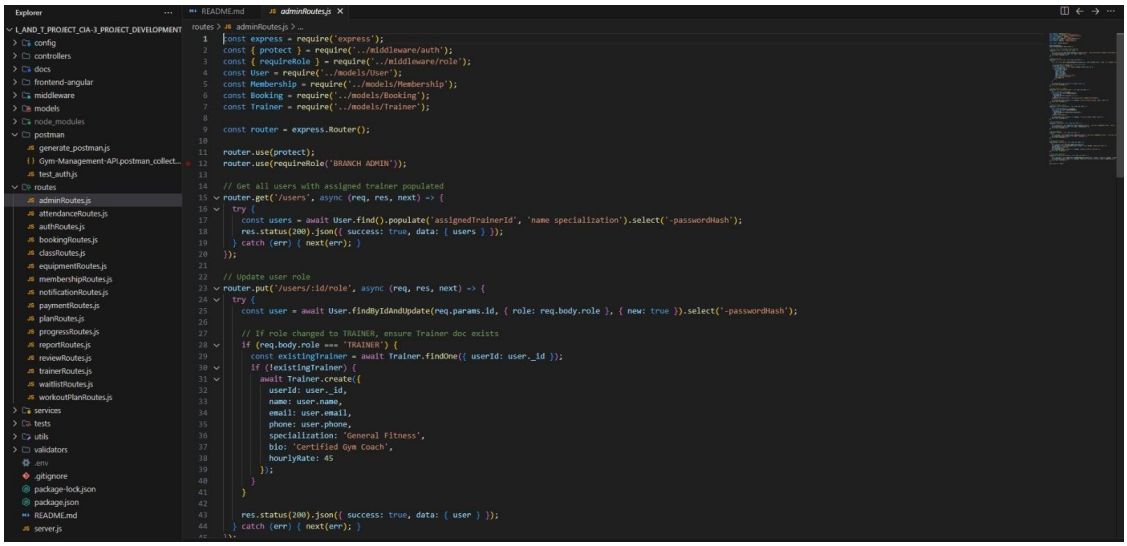

Fig. 14.2 – routes/adminRoutes.js — JWT + role-protected admin routes for listing users and updating roles

15. Screenshots of Final Output

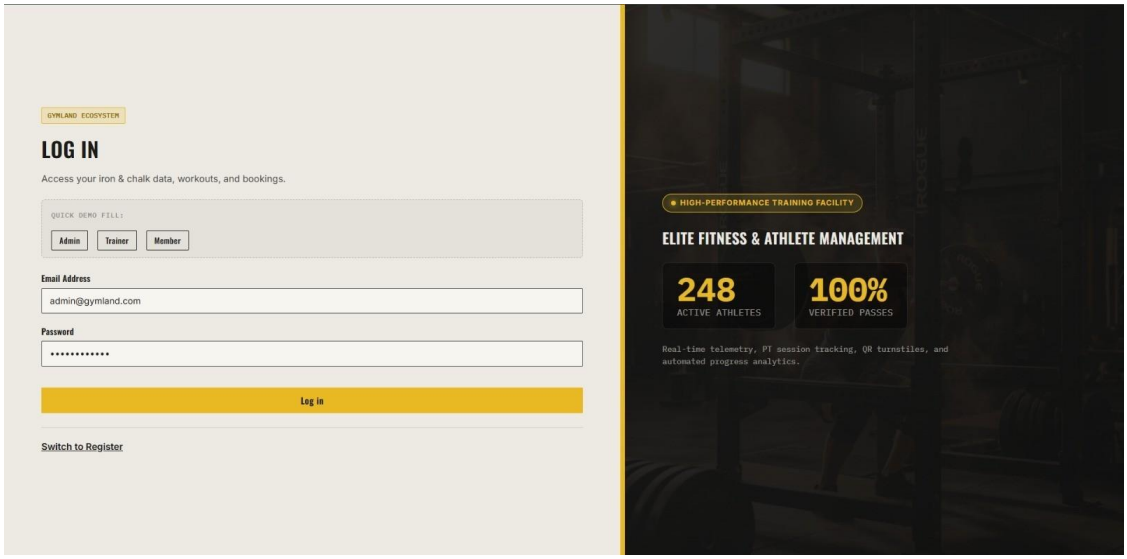

Fig. 15.1 – Login screen with quick demo-fill for Admin, Trainer, and Member roles

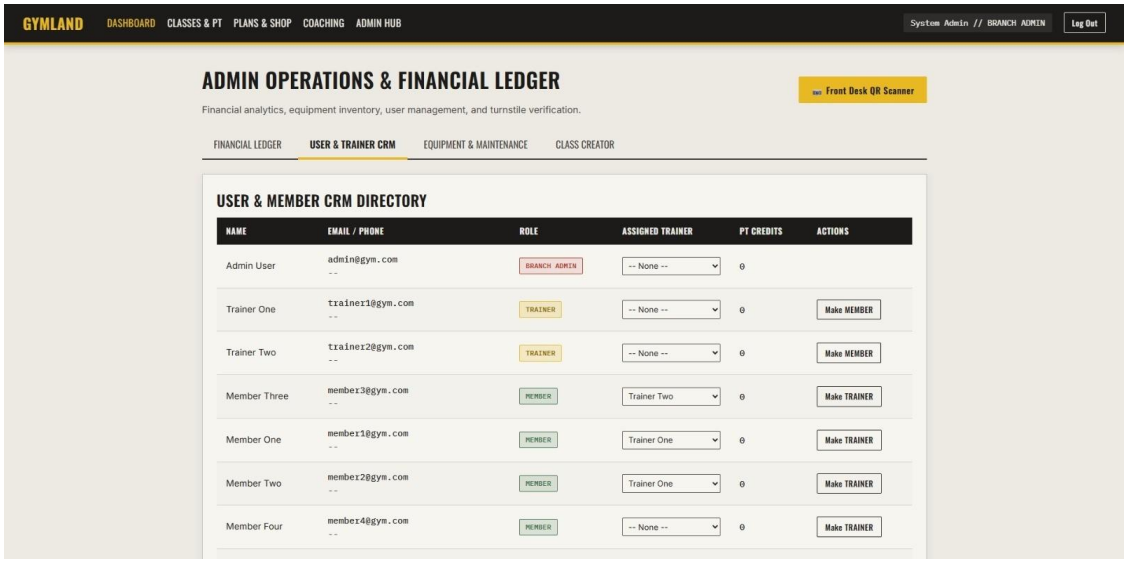

Fig. 15.2 – Admin dashboard: User & Trainer directory (role and trainer assignment management)

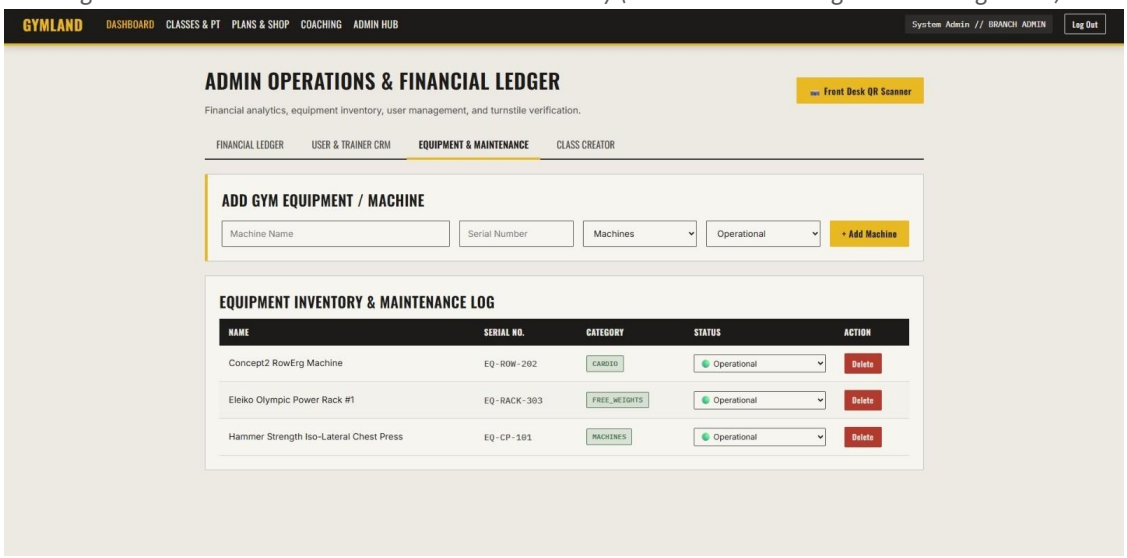

Fig. 15.3 – Admin dashboard: Equipment & maintenance inventory

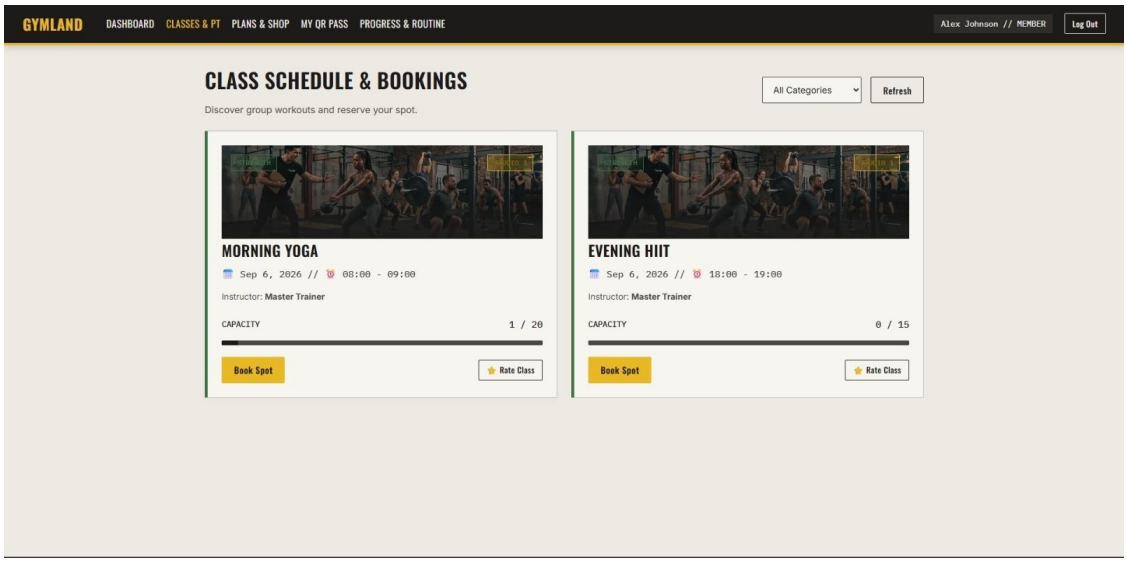

Fig. 15.4 – Member view: Class schedule and booking with live capacity tracking

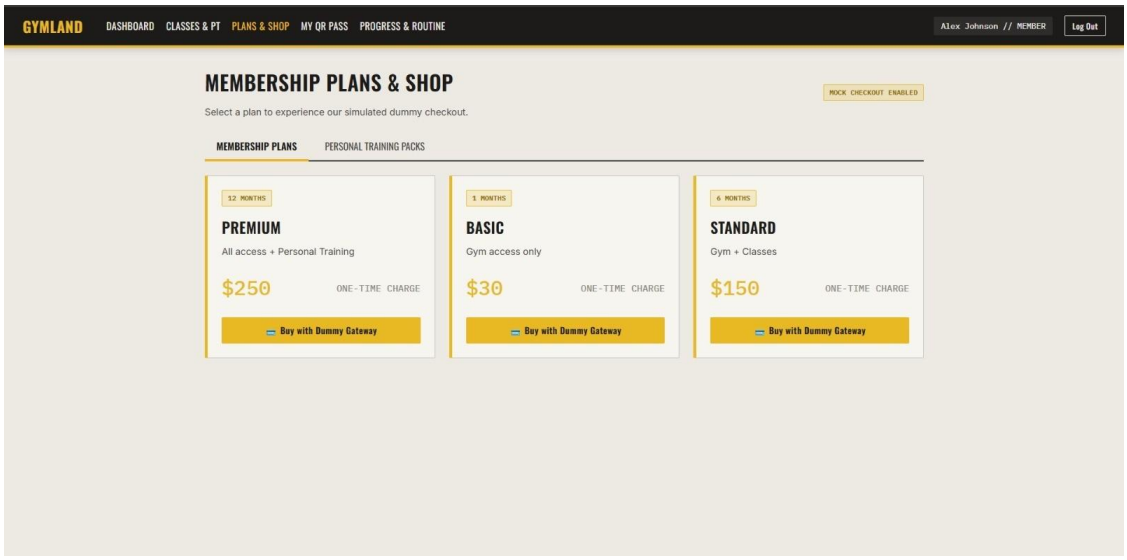

Fig. 15.5 – Member view: Membership plans and simulated checkout

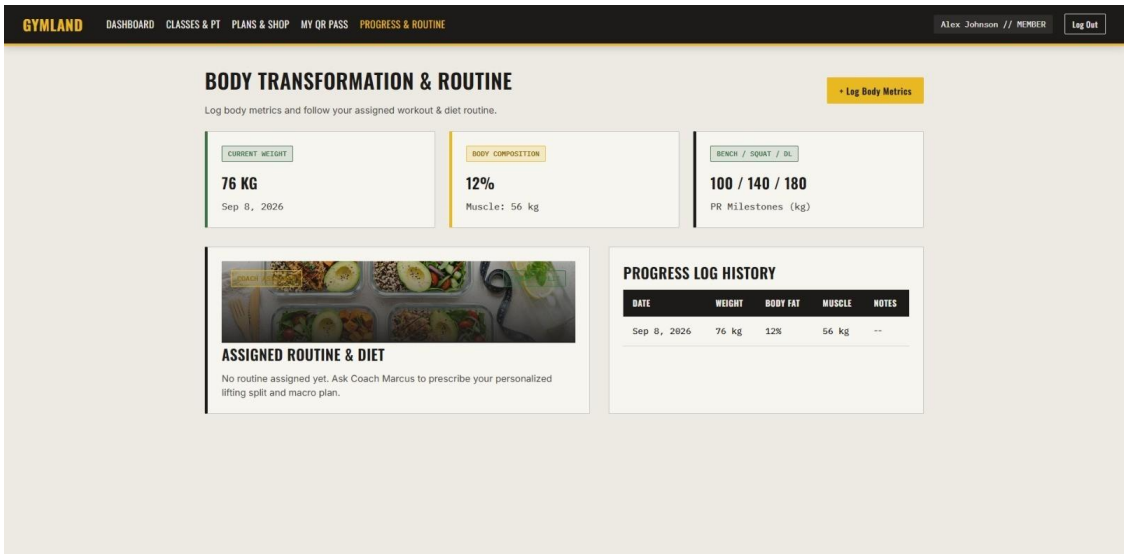

Fig. 15.6 – Member view: Progress & routine — body metrics and log history

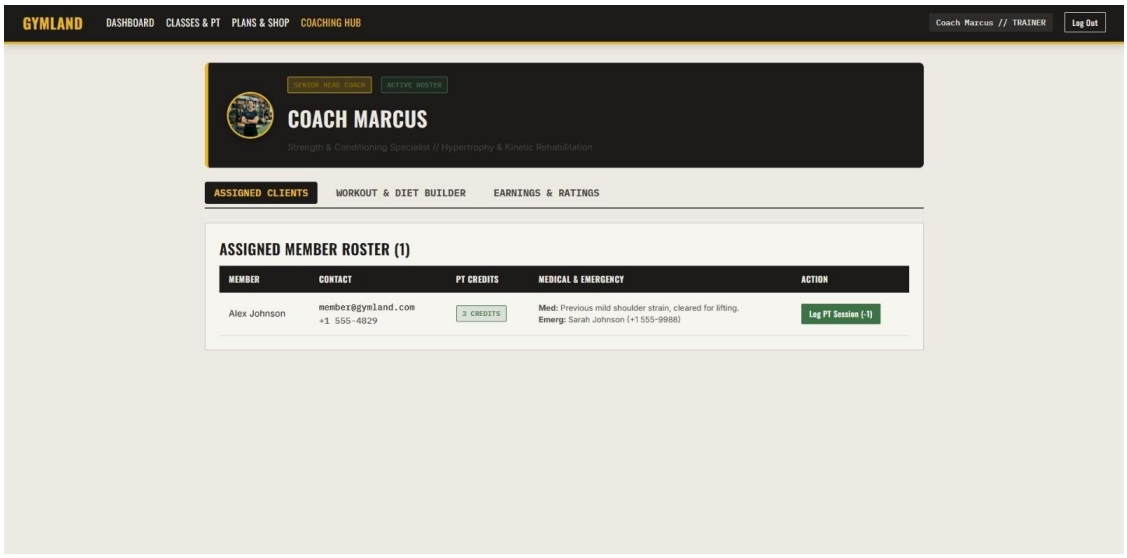

Fig. 15.7 – Trainer view: Coaching hub — assigned member roster

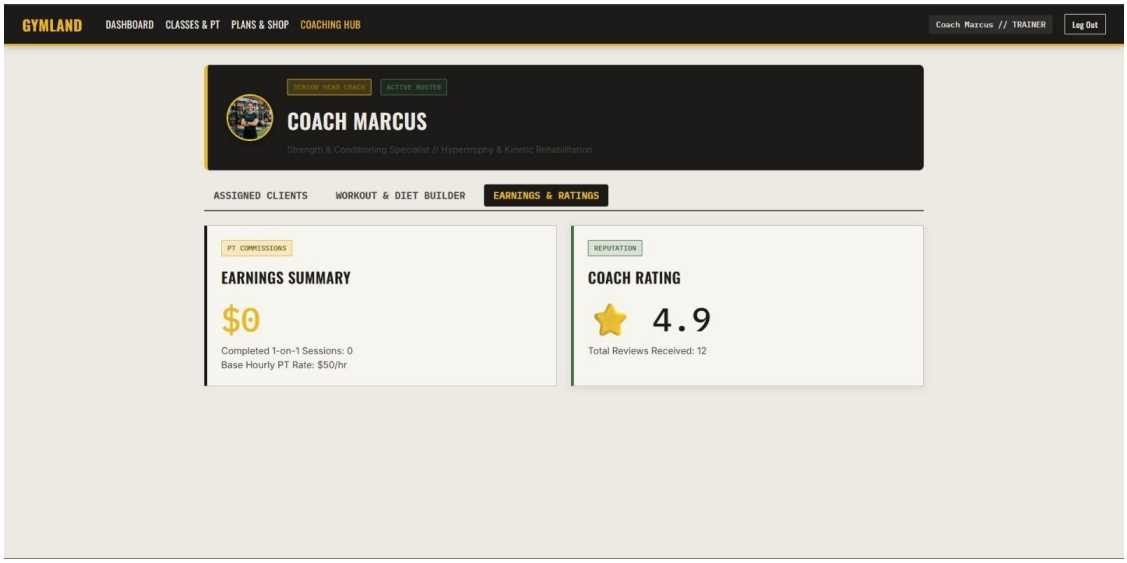

Fig. 15.8 – Trainer view: Coaching hub — earnings and ratings

16. Conclusion

The Gym & Fitness Membership Management System replaces manual, register-based gym administration with a role-based, API-driven platform that enforces real operational rules — membership expiry, class capacity, and waitlist promotion — as executable business logic rather than manual judgment calls. The layered architecture (routes → middleware → controllers → services → models) keeps the codebase maintainable, while server-side authorization ensures the role restrictions cannot be bypassed by a modified or malicious client.

16.1 Future Scope

- Real SMS/email delivery for notifications (currently in-app only).
- Multi-branch support with cross-branch reporting.
- Online payment gateway integration for membership purchase/renewal.
- Mobile app client consuming the same REST API.

17. References

- Node.js Documentation — https://nodejs.org/docs
- Express.js Documentation — https://expressjs.com
- MongoDB Documentation — https://www.mongodb.com/docs
- Mongoose ODM Documentation — https://mongoosejs.com/docs
- JSON Web Tokens (JWT) — https://jwt.io/introduction
- Postman Learning Center — https://learning.postman.com
- React Documentation — https://react.dev
- GitHub — https://github.com